CO-2 — Scenario-Based Question Bank

Pythonic Programming & Data Structures

Faculty-ready version: 25 application-oriented questions. Each numbered question is one complete 7–8 mark question. A specific **Coverage Note** is provided below every question to show the CO-2 concepts assessed.

Q1. E-Commerce Product Search — 8 Marks

An e-commerce company stores product information using lists, dictionaries and sets. Each product record contains a product ID, category, price and a set of tags. The development team currently uses nested loops to identify products belonging to a specified category and having at least one matching tag.
(a) Identify where a sequence, mapping and set would be most appropriate in this application. **[3]**
(b) Rewrite the filtering operation using appropriate Pythonic constructs such as comprehensions and set operations. **[3]**
(c) Explain why the selected data structures are preferable to repeatedly searching through lists. **[2]**

Coverage Note: Sequences, mappings, sets, comprehensions and performance

Q2. Railway Reservation System — 8 Marks

A railway reservation system maintains passenger records using a dictionary and maintains a set of passengers whose tickets have been confirmed. The existing system uses a list to check whether a passenger has been confirmed.
(a) Redesign the data representation using a dictionary and a set. **[3]**
(b) Write Python code to efficiently verify confirmation status and retrieve the passenger's booking details. **[3]**
(c) Compare the expected lookup performance of sets/dictionaries with that of lists. **[2]**

Coverage Note: Mappings, sets, Python data model and lookup performance

Q3. Data Analytics Pipeline — 8 Marks

A data analytics company processes one million transaction values each day. It must select transactions greater than ■10,000, apply a transformation and store the resulting values. A developer has implemented the operation using a traditional *for* loop.
(a) Implement the same operation using a Python list comprehension. **[3]**
(b) Compare the readability and performance of the loop and comprehension. **[2]**
(c) Explain a situation in which a conventional loop would be more appropriate than a comprehension. **[3]**

Coverage Note: Comprehensions, loops, Pythonic programming and performance

Q4. Video Processing Application — 8 Marks

A video-processing application receives millions of frames. Loading all frames into a list results in excessive memory consumption. The development team decides to process frames one at a time.
(a) Explain why a generator is more suitable than storing all frames in a list. **[2]**
(b) Write a generator function that reads and processes frames one at a time. **[3]**
(c) Explain how lazy evaluation reduces memory usage. **[2]**
(d) Mention one situation in which a list would still be preferable. **[1]**

Coverage Note: Generators, iterators, lazy evaluation and memory efficiency

Q5. Large-Scale Log Processing — 8 Marks

A cloud service receives a 50 GB log file every day. The existing program reads the complete file into memory before analysis, causing frequent memory failures.
(a) Design an iterator-based solution that processes the file line by line. **[3]**
(b) Differentiate between an iterable and an iterator. **[2]**
(c) Explain why the proposed solution is more memory efficient and state the role of *next()*. **[3]**

Coverage Note: Iterables, iterators, generators, next() and memory efficiency

Q6. Banking Database System — 8 Marks

A banking application opens a database connection to update customer transactions. When an exception occurs, the connection may remain open.
(a) Explain how a context manager can solve this problem. **[2]**
(b) Rewrite the database operation using the *with* statement. **[3]**
(c) Explain what happens when an exception occurs inside the *with* block and state one advantage over manual resource management. **[3]**

Coverage Note: Context managers, with statement, exceptions and resource management

Q7. Web Application Configuration — 8 Marks

A web application temporarily changes a configuration value while executing a particular operation. Developers repeatedly use *try-finally* blocks to restore the original value.
(a) Design a custom context manager for temporarily changing the configuration. **[4]**
(b) Demonstrate its use with a *with* statement. **[2]**
(c) Explain how the context manager guarantees restoration when an exception occurs. **[2]**

Coverage Note: Custom context managers, __enter__, __exit__, with statement and state management

Q8. Online Payment System — 8 Marks

An online payment service requires logging, authentication checks and execution-time measurement for several sensitive functions. The same code has been duplicated across functions.
(a) Explain why decorators are appropriate for this requirement. **[2]**
(b) Write a decorator that measures the execution time of a function. **[3]**
(c) Demonstrate applying the decorator to a payment function and state one advantage over duplicating the code. **[3]**

Coverage Note: Decorators, higher-order functions and code reuse

Q9. Food Delivery Application — 8 Marks

A food-delivery company needs to maintain the number of orders placed for each restaurant, an order queue, and the most frequently ordered food items. The current implementation uses ordinary dictionaries and lists for all three requirements.
(a) Identify suitable classes from the *collections* module for these requirements. **[3]**
(b) Write a Python implementation using the selected data structures. **[3]**
(c) Explain why these structures are more suitable than manually implementing the same behavior. **[2]**

Coverage Note: collections: Counter, defaultdict and deque

Q10. Social Media Analytics — 7 Marks

A social media platform needs to identify the most frequently used hashtags from one million posts.
(a) Explain why *collections.Counter* is suitable for this task. **[2]**
(b) Write Python code to count hashtag frequencies. **[3]**
(c) Demonstrate how to obtain the five most common hashtags. **[2]**

Coverage Note: collections.Counter and frequency counting

Q11. Data Processing Pipeline — 8 Marks

A data-processing company receives a stream of customer transactions. The system must group transactions by customer, generate combinations of selected products, and avoid creating unnecessary intermediate lists.
(a) Identify suitable functions from the *itertools* module. **[3]**
(b) Write a Python implementation demonstrating at least two appropriate *itertools* functions. **[3]**
(c) Explain how *itertools* can improve memory efficiency. **[2]**

Coverage Note: itertools, iterators and lazy processing

Q12. Employee Payroll System — 8 Marks

A payroll system calculates salaries using reusable functions for basic salary, tax and allowances. Some parameters remain fixed while employee-specific values change for each employee.
(a) Explain how *functools.partial()* can simplify this design. **[2]**
(b) Write a Python example using *partial()*. **[3]**
(c) Explain how function reuse can improve maintainability. **[3]**

Coverage Note: functools, partial and function reuse

Q13. Food Delivery Priority Queue — 8 Marks

A food-delivery application receives thousands of delivery requests. Emergency orders and premium-customer orders must be processed before normal orders.
(a) Select an appropriate Python data structure/module for implementing this priority queue. **[2]**
(b) Implement insertion and retrieval of the highest-priority order using *heapq*. **[4]**
(c) State the time complexity of insertion and removal operations. **[2]**

Coverage Note: heapq, priority queues and complexity

Q14. Hospital Emergency System — 8 Marks

A hospital maintains a queue of patients according to severity. A newly arriving critical patient must be served before patients with lower severity.
(a) Explain why a heap is appropriate for this problem. **[2]**
(b) Design a Python priority queue using *heapq*. **[4]**
(c) Explain how the implementation differs from a normal FIFO queue. **[2]**

Coverage Note: Heaps, priority queues and heapq

Q15. E-Commerce Price Search — 8 Marks

An e-commerce company maintains a sorted list of product prices and frequently needs to determine where a newly received price should be inserted without disturbing the sorted order.
(a) Explain why binary search is preferable to scanning the entire list for locating the insertion position. **[2]**
(b) Implement the insertion using the *bisect* module. **[3]**
(c) Explain the role of *bisect_left()* or *bisect_right()*. **[2]**
(d) State the limitation of insertion into a Python list even when *bisect* finds the position efficiently. **[1]**

Coverage Note: bisect, binary search, sorted lists and CPython list behavior

Q16. Cloud API Integration — 8 Marks

A cloud application receives data from multiple sources. Objects from different classes may provide a *read()* method, and the same processing function should work with all of them.
(a) Explain how duck typing can be used in this situation. **[2]**
(b) Write a Python function that accepts any object supporting *read()*. **[3]**
(c) Explain why explicit type checking may not be necessary and state one advantage and one limitation of duck typing. **[3]**

Coverage Note: Duck typing, dynamic typing and Python data model

Q17. Payment Gateway Integration — 8 Marks

An application communicates with multiple payment gateways. Some gateways may not provide a particular API method. One developer checks for the method before calling it, while another attempts the operation directly and handles the exception.
(a) Identify the two approaches as LBYL and EAFP. **[2]**
(b) Implement the EAFP approach in Python. **[2]**
(c) Explain when EAFP is preferable in Python. **[2]**
(d) Give one situation where LBYL may be more appropriate. **[2]**

Coverage Note: EAFP, LBYL, exceptions and duck typing

Q18. Stock Trading Platform — 8 Marks

A stock-trading platform must maintain unique stock symbols, stock details indexed by symbol, chronological transactions and high-priority trading requests.
(a) Choose an appropriate Python data structure for each requirement. **[4]**
(b) Justify each choice based on its access/search behavior. **[2]**
(c) Explain why using one data structure for all four requirements would be inefficient. **[2]**

Coverage Note: Lists, dictionaries, sets, heaps and data-structure selection

Q19. Python Application Performance — 8 Marks

A Python application processes 10 million records. The developer repeatedly concatenates strings inside loops and repeatedly performs membership searches in a list, resulting in poor execution time.
(a) Identify the two likely performance bottlenecks. **[2]**
(b) Suggest Pythonic alternatives for both problems. **[3]**
(c) Explain how CPython's implementation of built-in data structures can influence performance. **[2]**
(d) Explain why algorithmic complexity should be considered before micro-optimizing Python code. **[1]**

Coverage Note: Performance, CPython internals, built-in data structures and algorithmic complexity

Q20. Reusable Python Data-Structure Library — 8 Marks

A software company wants to create a reusable Python package containing implementations of Trie, Heap, Union-Find and Segment Tree. The package is intended for eventual publication on PyPI.
(a) Design the basic structure of the Python package. **[2]**
(b) Explain how type hints can improve usability and reliability. **[2]**
(c) Describe how unit tests should be organized for the data structures. **[2]**
(d) Mention two requirements for making the package suitable for PyPI publication. **[2]**

Coverage Note: Reusable utility library, algorithmic data structures, type hints, testing and packaging

Q21. Food-Delivery Order Processing System — 8 Marks

A food-delivery platform receives thousands of orders every minute. Each order contains customer information, restaurant information, food items and delivery priority. The system must identify unique restaurants, count orders, process high-priority orders first, and process incoming orders without loading the entire stream into memory.
(a) Select suitable Python data structures/modules for each requirement. **[3]**
(b) Implement priority-order processing using *heapq* and order counting using *collections*. **[3]**
(c) Explain how generators can be used to process incoming orders efficiently. **[2]**

Coverage Note: Integrated application of collections, heapq and generators

Q22. Identity Verification Data Pipeline — 8 Marks

An identity-verification service receives a very large stream of records. It must remove duplicate IDs, group records by region, validate incoming records and record processing time.
(a) Select appropriate Python data structures for duplicate detection and grouping. **[2]**
(b) Design a generator-based pipeline for processing records. **[2]**
(c) Use a decorator to measure processing time. **[2]**
(d) Decide whether EAFP or LBYL is more appropriate for validating incoming records and justify your answer. **[2]**

Coverage Note: Integrated application of sets, mappings, generators, decorators and EAFP/LBYL

Q23. Banking Transaction System — 8 Marks

A bank processes millions of transactions every day. Each transaction has an account number, timestamp and amount. The system must identify unique accounts, retrieve account details, process transactions in priority order and safely manage database resources.
(a) Choose suitable data structures for account lookup, unique accounts and transaction priority. **[3]**
(b) Explain how *heapq* can be used for priority transactions. **[2]**
(c) Explain how a context manager can safely manage database connections. **[2]**
(d) Mention one performance consideration when processing millions of transactions in CPython. **[1]**

Coverage Note: Integrated application of mappings, sets, heapq, context managers and performance

Q24. Large-Scale Log Monitoring System — 8 Marks

A cloud company analyzes a 100 GB server log every day. The system must identify error types, count their frequencies, process logs without loading the entire file into memory, and measure execution time.
(a) Explain why a generator or iterator is appropriate. **[2]**
(b) Use *collections.Counter* to identify frequent error types. **[2]**
(c) Design a decorator for measuring execution time. **[2]**
(d) Explain how the proposed design reduces memory consumption. **[2]**

Coverage Note: Integrated application of generators, iterators, Counter and decorators

Q25. E-Commerce Search Optimization — 8 Marks

An e-commerce website stores millions of product prices in sorted order. Customers frequently search for products within price ranges, while new products are continuously added.
(a) Explain how *bisect* can be used for efficient position or range searching. **[2]**
(b) Write Python code using *bisect_left()* and *bisect_right()* to identify products within a given price range. **[3]**
(c) Explain the difference between finding an insertion position and actually inserting into a Python list. **[2]**
(d) Suggest an alternative approach if insertions become extremely frequent, with justification. **[1]**

Coverage Note: Integrated application of bisect, binary search, sorted lists and performance considerations

Overall coverage: The set covers sequences, mappings, sets, comprehensions, loops, iterators, generators, context managers, decorators, collections, itertools, functools, heapq, bisect, duck typing, EAFP/LBYL, CPython/performance considerations, reusable data-structure libraries, type hints, testing and packaging.